# Integration & Release Plan

Online Multiplayer Game Platform
SENG 300 Winter 2026 Group 8

Justin Ma
Integration & Release Lead

Last Updated: 2026-02-27

## Contents

# 1 Introduction

This plan describes how code moves from individual branches into a stable `main` branch, and how we package and verify the project before the deadline. Everyone on the team needs to follow the shared procedures in this document. Our purpose on the Integration & Release (I&R) sub-team is to handle the coordination work, which we will describe in the sections below.

# 2 Team Structure and Responsibilities

## 2.1 Integration & Release Sub-Team

We have three members, each with a primary focus area so that ownership and Git contributions are clear. That said, we will cover for each other as needed.

| Primary Owner | Contact Emails | Primary Focus |
|---|---|---|
| Justin Ma | justin.ma3@ucalgary.ca | Cross-team coordination, merge oversight, interface reviews, integration issue tracking |
| Dai Toan Dang | daitoan.dang@ucalgary.ca | Build and release: README maintenance, release dry-runs, build verification |
| Anh Tuan Vo | anhtuan.vo@ucalgary.ca | Documentation: CHANGELOG, CURRENT_STATE updates, integration health reports |

All three of us participate in merge reviews and integration sessions.

**Note on Identity Management:** During P1, our sub-team helped with Identity Management and Authentication design since we had extra capacity. Going forward, we may hand this off to another sub-team so we can focus on our core integration and release responsibilities. We'll coordinate the handoff with the relevant team lead and the TA.

**Integration and Release Sub-team Responsibilities**

- **Main branch stability**: Making sure `main` always compiles and runs. If a merge breaks it, we coordinate the revert and communicate the fix to everyone.
- **Merge request oversight**: Reviewing, verifying, and executing all merges into `main`, including running a build verification after each merge.
- **Build and run instructions**: Keeping the README accurate and verifying it through fresh-clone dry-runs before the deadline.
- **Documentation**: Maintaining doc files (for example: README, CHANGELOG, CURRENT_STATE, and INTEGRATION_ISSUES) so they reflect the actual state of the project.
- **Cross-team coordination**: Scheduling merge windows, managing merge order, and helping sub-teams resolve integration conflicts.
- **Release packaging**: Tagging the release, running the final dry-run, and making sure the repository is ready for submission.
- **Integration issue tracking**: Keeping a log of integration issues and how they were resolved for the P2 group deliverable.

### 2.2 Developer Responsibilities

No matter which sub-team you're on, you need to:

1. Create feature branches following the naming convention.
2. Write clear commit messages in the agreed format.
3. Run the full build-test-launch checklist before opening a merge request.
4. Fill out the MR template completely.
5. Respond to code review feedback in a timely manner.
6. Update relevant documentation when your MR changes build instructions, interfaces, or user-facing behaviour.
7. Report blockers and attend cross-team meetings when asked.

### 2.3 Sub-Team Lead Responsibilities

Each sub-team lead is responsible for making sure their team follows branch naming conventions, flagging upcoming merges and dependencies to the Integration Lead, doing a first-pass review on their team's MRs before sending them to I&R, and attending (or sending a rep to) cross-team meetings.

## 3 Branching Strategy

### 3.1 Feature-Branch Workflow

We will use a **feature-branch workflow**, which is popular. This means that short-lived branches merge directly into a protected `main` branch through merge requests. We will *NOT* use long-lived develop branches or separate release branches. We keep it simple because we don't have CI/CD and most of us are still getting comfortable with Git workflows.

### 3.2 Branch Types

**`main` (Protected)**:

1. The single source of truth.
2. Must always compile and run.
3. No direct pushes.
4. All changes enter through merge requests.

**Feature branches**:

1. Short-lived (aim for a week max).
2. One feature or fix per branch.
3. Created from `main` and merged back via MR.
4. Keep branches after merging for grading purposes.

### 3.3 Naming Branches

**Format:** `<team-prefix>/<type>/<issue#>-<short-description>`

**Common Type Prefixes:**

| Type | Use When |
|---|---|
| `feature/` | Adding new functionality |
| `bugfix/` | Fixing a defect |
| `refactor/` | Restructuring code without changing behaviour |
| `docs/` | Documentation-only changes |
| `test/` | Adding or updating tests |

**Rules for Naming Branches:**

1. All lowercase.
2. Use hyphens as word separators.
3. Include the GitLab issue number.
4. Keep it under 50 characters.

### 3.4 General Reminders About Branches

- 5-day maximum lifespan: If a branch lives longer than 5 days, it needs to either be merged or closed with an explanation.
- Stay synced with main: Merge `main` into your feature branch when you start work and at least once every 3 days to minimize conflicts.
- Don't rebase pushed branches: Once a branch is on the remote, use `git merge`, not `git rebase`.
- One logical change per branch: Don't bundle unrelated changes together.

### 3.5 Branch Protection

If possible, the I&R team will try to configure these standard protection rules in GitLab:

| Setting for `main` | Value |
|---|---|
| Allowed to merge | Maintainers only |
| Allowed to push and merge | No one |
| Allowed to force push | Disabled |
| Minimum approvals required | 1 |

| Setting for `main` | Value |
|---|---|

# 4 Commit Standards

## 4.1 Message Format

```
[Team] Short description (#issue-number)
```

The team tag should match your sub-team: `[Platform]`, `[Client]`, `[Rules]`, `[Quality]`, or `[Integration]`. For example:

```
[Platform] Refactor room state to use enum instead of strings (#55)

Strings were causing silent bugs when typos crept in. Enum gives us
compile-time safety and makes the state machine easier to reason about.
```

# 5 Merge Request Workflow

## 5.1 Create a Merge Request

**Every change to `main` goes through a Merge Request.**

**Rules for Merge Requests:**

1. If you want feedback on your work from others, open an MR as soon as you have a working branch and prefix the title with `Draft:`.
2. Aim for 1–2 MRs per developer per week.
3. Create the MR as soon as the branch is pushed so the rest of the team can see what's coming.

## 5.2 Merge Request Description Template

Every MR must use the following standard template. If possible, we'll try to store this as a template in GitLab so it auto-fills when you create an MR.

```
## Summary

<!-- What does this MR do? 2-3 sentences. -->

## Related Issue(s)

<!-- Link to GitLab issues. Use "Closes #XX" for auto-closure. -->

Closes #

## Type of Change

- [ ] Feature
- [ ] Bug fix
- [ ] Refactor
- [ ] Documentation
- [ ] Test
- [ ] Build
- [ ] Other (please describe):
```

```
## Sub-Team

- [ ] Platform Core
- [ ] Client & UI
- [ ] Rules & Validation
- [ ] Quality & Testing
- [ ] Integration & Release

## Changes Made

<!-- What specifically changed? -->

-

## Testing Done

- [ ] `./mvnw clean compile` passes
- [ ] `./mvnw test`: all tests pass
- [ ] `./mvnw javafx:run` launches without crash
- [ ] Manually tested the affected feature(s)

## Pre-Merge Checklist

- [ ] Branch is up to date with `main`
- [ ] No commented-out code or debug print statements
- [ ] Public methods have Javadoc comments
- [ ] README updated (if build/run instructions changed)
- [ ] CHANGELOG [Unreleased] section updated
```

### 5.3 Merge Request Labels and Milestones

We use the main GitLab Issues board for all project tracking. For integration visibility, label each MR with your team name and the change type. Assign every MR to the `P2` milestone so we can track progress toward the deadline.

## 6 Code Review

Everyone is expected to keep code review discussions constructive and professional. Authors should self-review their own diff before requesting review, and respond to all feedback in a timely manner.

Try to use prefixes in your comments to keep review clear. For example:

- `Blocking:` for things that must be fixed
- `Nit:` for optional suggestions
- `Question:` when you're just trying to understand something

### 6.1 Who Reviews What

| MR Type | Required Reviewers |
|---|---|
| Standard change within one sub-team | 1 peer from the same sub-team |
| Change to shared interfaces | 1 peer + 1 reviewer from each affected sub-team |

| MR Type | Required Reviewers |
|---|---|
| Documentation-only | 1 peer from any sub-team |
| I&R changes (build, README, etc.) | 1 I&R member + 1 reviewer from any other team |

### 6.2 What Reviewers Should Look For

These criteria align with what the TA focuses on in their reviews:

1. **Correctness**: Does the code do what the MR description says?
2. **Architecture alignment**: Does it follow the platform-first model and use agreed interfaces?
3. **Readability**: Would another developer understand this code?
4. **Integration safety**: Could this break another sub-team's code?
5. **Test coverage**: Are there tests for the happy path and at least one edge case?
6. **Documentation**: Are public methods documented? Are relevant docs updated?

## 7 Build and Verification

### 7.1 Build System

All of this comes from information in the starter files that the course instructor provided:

| Component | Details |
|---|---|
| Build tool | Maven |
| JDK | **JDK 25** |
| UI framework | JavaFX |
| Base package | `ca.ucalgary.seng300` |
| Compile | `./mvnw clean compile` (or `mvnw.cmd clean compile` on Windows) |
| Run | `./mvnw javafx:run` (or `mvnw.cmd javafx:run` on Windows) |
| Test | `./mvnw test` |

The Maven Wrapper downloads the correct Maven version automatically. JDK 25 is the only prerequisite you need to install manually.

> **Don't run MainApp directly** through IntelliJ's green Run button. Always use `./mvnw javafx:run` or IntelliJ's Maven tool window → Plugins → javafx → javafx:run.
>
> **Note on testing:** The starter `pom.xml` doesn't include a test framework yet. Once the Quality & Testing team adds JUnit 5, `./mvnw test` will run the test suite. Until then, `./mvnw test` will succeed but run zero tests.

### 7.2 Pre-MR Verification (Every Developer)

Before opening a merge request, do the following:

1. Pull latest `main` and merge it into your branch: `git fetch origin main && git merge origin/main`
2. Run a clean build: `./mvnw clean compile`: expect `BUILD SUCCESS`
3. Run all tests: `./mvnw test`: expect all tests to pass
4. Launch the app: `./mvnw javafx:run`: expect the JavaFX window to open without crashing
5. Manually verify that your change works as described in your MR

**If any step fails, don't open the MR.** Fix the issue first.

### 7.3 Post-Merge Verification (I&R Team)

After every merge to `main`, someone on the I&R team pulls `main`, runs the compile-test-launch sequence, and walks through the smoke test checklist below. If anything fails, we initiate the broken-main protocol.

### 7.4 Smoke Test Checklist

This checklist verifies platform-level flows from all sub-teams. Items 1–3 are the minimum, and we add the other items as we implement more features.

| # | Check | Expected Result |
|---|---|---|
| 1 | Clean compile | `BUILD SUCCESS` |
| 2 | All tests pass | No failures |
| 3 | Application launches | JavaFX window opens |
| 4 | Login / profile screen | Identity management UI renders |
| 5 | Lobby / game selection | Game list is visible |
| 6 | Screen navigation | No crashes when switching between screens |
| 7 | Matchmaking flow | Can search for or queue into a match |
| 8 | Start Tic-Tac-Toe | Board renders and turns alternate correctly |
| 9 | Start Connect Four | Board renders and piece drops work correctly |
| 10 | Start Checkers | Board renders and piece movement works |
| 11 | Leaderboard display | Scores render |
| 12 | In-game chat | Basic messages can be sent and displayed |
| 13 | Exit application | Window closes cleanly without hanging |

## 8 Definition of Done

A merge request is considered “done” only when all of the following are true:

#### Code

- Compiles without errors or warnings.
- Follows Java naming conventions.
- No commented-out code, debug prints, or leftover TODOs.
- Public methods and classes have Javadoc comments.

#### Testing

- All existing tests pass.
- New code includes tests covering the happy path and at least one edge case.

#### Integration

- Branch is merged with the latest `main`.
- Full project builds and launches successfully.

#### Review

- MR description is complete.
- At least one approval from a qualified reviewer.
- All reviewer comments are resolved.

### Documentation

- Interfaces are documented.
- README updated if build/run instructions changed.

# 9 Integration Scheduling

## 9.1 I&R Deliverable Responsibilities

These are the group deliverables from the project document that the I&R sub-team either owns or coordinates.

| Deliverable | I&R Role | Owner within I&R |
|---|---|---|
| Clear build and run instructions | **Own** | Dai Toan Dang |
| Architecture and project structure overview | **Coordinate** | Justin Ma |
| Updated design artifacts (reflecting implementation) | **Coordinate** | Justin Ma |
| Change summary of major changes | **Own** | Anh Tuan Vo |
| How integration issues were resolved | **Own** | Justin Ma |
| Progress summary (what's implemented, what remains) | **Coordinate** | Anh Tuan Vo |
| Final release tag and verification | **Own** | Dai Toan Dang |

## 9.2 Integration Day Procedures (I&R)

During weekly integration sessions:

1. Each sub-team pushes their latest work to their feature branches.
2. I&R reviews all open MRs for readiness.
3. We merge in priority order.
4. After each merge, we run the post-merge verification.
5. Any failures are either fixed immediately or reverted.
6. We update the integration log and post status to Discord.

## 9.3 Shared Interface Management

Platform Core will be responsible for managing Java interfaces and data models that are shared across sub-teams. This means that we will never have one team directly depend on another team's implementation package. This way, we can minimize merge conflicts and allow teams to work in parallel as much as possible.

Based on P1 design work, shared interface areas are expected to include:

1. Database and persistence stubs
2. Identity and user profile contracts
3. Game registry interfaces
4. Matchmaking interfaces
5. Turn engine contracts
6. Move validation interfaces
7. Leaderboard/scoring contracts

## 9.4 Merge Priority Order

When multiple MRs are ready at the same time, we merge in this order to minimize cascading conflicts:

1. **Platform Core**: Data models, database stubs, room lifecycle, and turn engine form the foundation.

2. **Rules & Validation**: Move validation and scoring depend on Platform Core's data models.
3. **Client & UI**: Screens, board rendering, and UI flows depend on Platform Core interfaces and may display validated data from Rules & Validation.
4. **Quality & Testing**: Integration tests exercise flows across the above; merging tests last avoids churn.
5. **Integration & Release**: Documentation and build config carry the lowest conflict risk.

Within the same priority level, smaller MRs go first.

# 10 Release Management

## 10.1 Versioning

We keep versioning simple:

| Phase | Tag | Meaning |
|---|---|---|
| Early development builds | `v0.x.y` | Incremental progress snapshots |
| Final submission | `v1.0.0` | The platform is stable and ready to demo |

## 10.2 Git Tags

We use annotated tags for releases:

```
git tag -a v1.0.0 -m "P2 Final Release: integrated multiplayer game platform"
git push origin v1.0.0
```

Don't move or delete tags once they're created.

## 10.3 Release Readiness Checklist (I&R)

Before submission, run through this:

- ☐ All merge requests reviewed, approved, and merged
- ☐ No unresolved critical issues in the `P2` milestone
- ☐ All tests pass
- ☐ Application launches and runs without errors
- ☐ Demo scenarios tested end-to-end
- ☐ README.md is accurate and complete
- ☐ CHANGELOG.md is up to date
- ☐ CURRENT_STATE.md feature matrix is accurate
- ☐ Git tag created
- ☐ Release dry-run passed

## 10.4 How to Do a Dry-Run

Before we submit the project for grading, we will do a "release dry-run" to verify it actually works.

1. Clone fresh to a clean directory (not your working copy).
2. Check out the release tag: `git checkout v1.0.0`
3. Follow the README instructions exactly.
4. Build: `./mvnw clean compile`
5. Run: `./mvnw javafx:run`

6. Walk through every demo scenario and verify features listed in CURRENT_STATE.md as "Working" actually work.
7. If anything fails, either fix the code/docs or update CURRENT_STATE.md honestly.
8. Record the result (pass/fail, date, who ran it).

# 11 Challenges and How We Handle Them

Here are the main challenges we're anticipating and our plans for dealing with them.

### Last-Minute Merges

This is probably our biggest risk, because if everyone merges at the same time right before the deadline, we won't have time to resolve conflicts or fix broken builds. Hopefully, having integration sessions will help. We'll also be posting integration updates on Discord to keep everyone aware of what's been merged and what's still outstanding.

### Incompatible Interfaces

If implementations don't match the shared contracts, things will break when teams try to integrate. That's why we require cross-team review for any changes to shared interfaces and why we merge Platform Core first.

### Main Breaks

We have already outlined a procedure above for how to verify the build after every merge. If something does break:

1. We communicate with everyone on Discord `#emergency`.
2. Identify the breaking commit.
3. Revert it using `git revert`.
4. Verify the build again.
5. Communicate the fix.

# 12 Communication and Escalation

## 12.1 Meetings

| Meeting | Frequency | Duration | Attendees | Purpose |
|---|---|---|---|---|
| I&R Internal Sync | 2×/week | 15 min | Justin, Dai, Tuan | Internal coordination |
| Cross-Team Integration Sync | 1×/week | 15 min | 1 rep per sub-team + I&R | Cross-team status and blockers |

## 12.2 Discord Channels

Discord is already our main communication tool. We use `#general` for general discussion and announcements and `#emergency` for broken main or critical blockers. GitLab MR and issue comments are used for code review and task-specific discussion.

### 12.3 Escalation

```
Level 1: Within your sub-team
  ↓ If unresolved
Level 2: Contact the blocking team's lead
  ↓ If unresolved after 24 hours or if the deadline is soon
Level 3: Integration Lead (Justin Ma)
  ↓ If architectural or project-level or in an emergency when main is broken
Level 4: TA / Product Owner
```

## Acknowledgements

Before starting this document, I needed inspiration and guidance on how to structure and format it, including sections and sub-sections to include. So, I used AI-assisted deep research to find the best practices and strategies for integration and release in 2026.

From those structures and formats I found, I decided what information I thought would be relevant to our specific project. Then, I wrote the content of each section myself based on my understanding of our project and the best practices I researched.